

Databasutveckling och lagring — Deliverable

2026-02-04

1 Intro

This is the final project for the course *Databasutveckling och lagring*. It is a group project.

2 Groups

The groups will be created when the project is introduced. This document will be updated with a list of groups afterwards.

2.1 It's not easy working in group

The knowledge level in the class is varied, and it will be so in the group, too. While it will not be possible for everyone in the group to contribute *exactly* as much as everybody else, the following points are important:

1. **Everyone should get the possibility to actively participate in the project.** So don't let one or two members in the group fix everything.
2. **Everyone should actively participate in the project**, so if there is any members who aren't active, let me know as soon as possible.

A good strategy could be to divide all the tasks in the projects among the members, so everyone does something, nobody does everything and nobody does nothing.

Practical planning is left up to the group.

2.2 Groups

2.2.1 Group 1

- Said
- Alexander
- Axel
- Mika
- Cebraail

2.2.2 Group 2

- Mikhael

- Dennis

- Tousif

- Kabir

2.2.3 Group 3

- William

- Lisa

- Andy

- Levie

- Baraa

2.2.4 Group 4

- Christian
- Victor F
- Vincent
- Fatimah
- Emil

2.2.5 Group 5

- Adam
- Reza
- Nenne
- Angelica
- Sam

2.2.8 Group 8

- Thomas
- Patrik
- Wivine

2.2.6 Group 6

- Veronica
- Uma
- Viktor
- Anton
- Azal

2.2.7 Group 7

- Mohamed Hersi
- Marcus
- Simon
- Elnor
- Morad

3 Deadline

The deadline for the project is Friday **2026-02-27 at 12:00**.

4 What to turn in

All you need to turn in is a **link to a GitLab repository** containing all the needed files.

5 Requirements

You have all been hired by a startup that is building the next generation online platform for pet owners.

Before our devs can get to work, they need you to come up with a schema for the main production database.

You need to deliver one or more `.sql` files that **create the needed tables, views, keys and relations**, as well as **populate the tables with test data**, so we can test our applications while we develop them.

The database design should be normalized to 3NF.

After endless meetings with people from sales, social, research and marketing teams, you've come up with the following requirements.

5.1 Database

5.1.1

We will use **MariaDB** version 12 for the production database.

5.2 Users

5.2.1

Users sign up for our services by choosing a unique username and password, as well as providing a unique email address they need to confirm before their account can be used.

5.2.2

When a user buys something from our app's store, they need to provide a billing address and a delivery address.

5.3 Pets

5.3.1

Users can register their pets on our platform. A pet can be one of many different species. When choosing the specie, the user should be able to choose a specie using either the common name (i.e.: *dog*) or the latin name (i.e.: *Canis Familiaris*). Either names should result in the same specie being chosen.

5.3.2

A pet can also have a given name and a date of birth (so that we can send a happy birthday message).

5.3.3

Users can also write a description of their pets for other users to read.

5.3.4

A pet can be a current pet (alive) or a former pet (dead).

5.4 Shop

5.4.1

Our webshop will offer many products, divided into different categories.

Each product has a unique SKU (https://en.wikipedia.org/wiki/Stock_keeping_unit), a name, a manufacturer whom we buy it from, a buying price and a selling price.

Each manufacturer we buy our products from has a name, an adress and a contact person (along with their contact information).

5.4.2

Products have a descriptive text where further details are stored, such as sizes, colours, materials, instructions, etc..

5.5 Orders

5.5.1

An order is submitted by a user, and it contains one or more products in different quantities.

An order has a status, and it can be awaiting, fullfilled or cancelled.

5.6 Warehouses

5.6.1

We will open warehouses in the country's major cities to allow quick fulfillment and delivery times across the whole country.

Each order will be assigned to a warehouse to be fullfilled.

5.6.2

We will need to keep an inventory for all the products across all warehouses.

5.7 Social

5.7.1

Users can send messages to each other. Each message has a sender and a receiver.

5.7.2

A message can either be an answer to another message (which we say is the parent of the message), or the beginning of a new thread (without a parent).

5.8 For VG

5.8.1

All changes (INSERT, UPDATE and DELETE) on all tables are to be logged in separate log tables using TRIGGERS (1 log table per table).

5.8.2

A python script is to be provided in the repo. It should allow the user to chose a warehouse from the ones present in the DB, and it should show if all the current (non-fulfilled and non-cancelled) orders assigned to that warehouse can be fulfilled with the current inventory present in the location.

If it is not possible to fulfill all orders, the script should print out a list of which and how many products are missing at each warehouse.

5.8.3

Make your design work with species having more than one common name, and more than one latin name.